

Haute Disponibilité (HAProxy / Keepalived) et Déploiements Rolling-Update Zéro-Downtime

Un serveur isolé constitue une vulnérabilité critique :

- **Le point unique de défaillance (*Single Point of Failure - SPOF*)** : Si la machine physique tombe en panne matérielle, si le noyau panique ou si une mise à jour impose un redémarrage, le service web est instantanément coupé pour l'intégralité des clients.
- **L'impératif contractuel des SLA (*Service Level Agreements*)** : En environnement bancaire, e-commerce ou industriel, les engagements de disponibilité imposent un taux de service supérieur à 99.9% (trois neufs) voire 99.99% (quatre neufs) :

$$\text{Disponibilité (A)} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

Taux de disponibilité contractuel	Temps d'arrêt maximal toléré par an	Temps d'arrêt maximal toléré par mois
99% (Deux neufs)	3 jours 15 h 39 min	7 h 18 min
99,9% (Trois neufs)	8 h 45 min	43 min 49 s
99,99% (Quatre neufs)	52 min 35 s	4 min 23 s
99,999% (Cinq neufs)	5 min 15 s	26 s

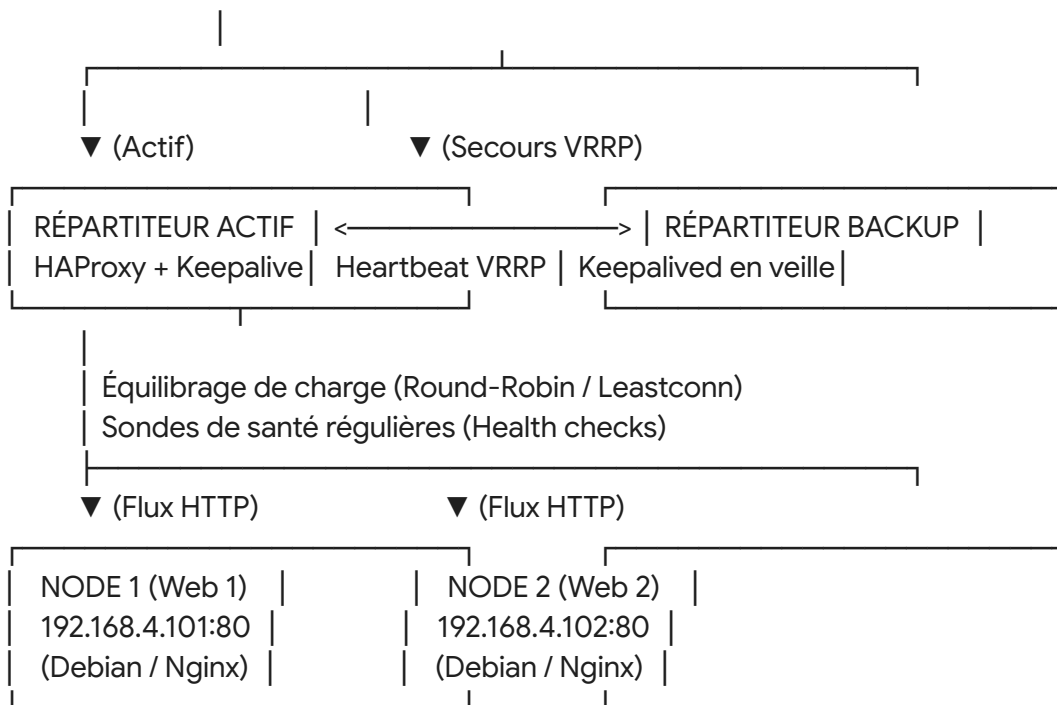
Pour éliminer les SPOF et garantir un SLA élevé, l'infrastructure doit mutualiser les ressources via la **répartition de charge (*Load Balancing*)** et la **redondance de passerelle (*Failover IP*)**.

CLIENTS EXTERNES (INTERNET / INTRANET)



ADRESSE IP VIRTUELLE (VIP : 192.168.4.200)

Portée par le nœud MASTER actif



2. Répartition de charge avec HAProxy : Architecture et mécanique interne

Créé par Willy Tarreau, **HAProxy (High Availability Proxy)** est le standard mondial de la répartition de charge logicielle haute performance sous Linux. Capable de traiter plusieurs centaines de milliers de requêtes simultanées avec une consommation mémoire minimale, il opère selon deux modes fondamentaux.

A. Équilibrage Couche 4 (Transport TCP) vs Couche 7 (Applicatif HTTP)

1. Mode Couche 4 (mode tcp) :

- HAProxy ouvre une socket vers le client et une socket vers le backend, puis transfère les paquets TCP bruts de façon bidirectionnelle sans inspecter les en-têtes applicatifs.
- *Avantages* : Débit ultra-rapide, très faible charge CPU.
- *Inconvénients* : Impossible de router selon l'URL, les cookies ou d'injecter des en-têtes HTTP.

2. Mode Couche 7 (mode http) :

- HAProxy analyse intégralement la requête HTTP (méthode, URL, en-têtes Host, User-Agent, cookies).
- *Fonctionnalités avancées* : Terminaison SSL/TLS, réécriture d'en-têtes (X-Forwarded-For, X-Forwarded-Proto), persistance de session (*Sticky Sessions*) et

routage conditionnel par expressions régulières (ACLs).

B. Algorithmes de répartition de charge

HAProxy arbitre la distribution des requêtes selon des politiques configurables par backend :

- **roundrobin (Tour de rôle)** : Les requêtes sont transmises séquentiellement à chaque serveur actif ($S_1 \rightarrow S_2 \rightarrow S_1 \rightarrow S_2$). Adapté aux serveurs de puissance homogène traitant des requêtes à durée d'exécution équivalente.
- **leastconn (Connexions minimales)** : La nouvelle requête est dirigée vers le serveur qui traite actuellement le plus petit nombre de connexions actives. Idéal pour les charges hétérogènes (requêtes SQL lourdes, transferts de gros fichiers, sessions WebSocket).
- **source (Hachage d'IP source)** : L'adresse IP du client est hachée pour désigner toujours le même serveur backend. Permet d'assurer une persistance de session basique sans modification applicative.

C. Surveillance de santé active (Active Health Checking)

HAProxy ne se contente pas d'envoyer les requêtes en aveugle ; il audite périodiquement chaque nœud de production :

server node1 192.168.4.101:80 check inter 2000 fall 3 rise 2

- **inter 2000** : Envoi d'une sonde de test toutes les $2\,000\text{ ms}$ (2 secondes).
- **fall 3** : Il faut 3 échecs consécutifs pour que le serveur soit déclaré DOWN et retiré de la distribution de trafic.
- **rise 2** : Après rétablissement, il faut 2 succès consécutifs pour que le serveur soit réintégré dans le pool (UP).
- **Sonde applicative avancée (httpchk)** : Plutôt qu'un simple ping TCP sur le port 80, HAProxy exécute une vraie requête HTTP (option httpchk GET / HTTP/1.1\r\nHost:\localhost) et n'accepte le serveur que s'il répond un code HTTP 2xx ou 3xx.

3. Redondance de passerelle avec Keepalived et le protocole VRRP

Si HAProxy protège les serveurs web node1 et node2, que se passe-t-il si la machine hébergeant HAProxy tombe elle-même en panne ? Le répartiteur devient le nouveau point unique de défaillance.

Pour neutraliser ce risque, on déploie une paire de répartiteurs redondants fédérés par **Keepalived**.

A. Le protocole VRRP (*Virtual Router Redundancy Protocol* - RFC 5798)

VRRP permet à plusieurs machines physiques ou virtuelles de partager une même adresse IP virtuelle (**VIP - Virtual IP**) :

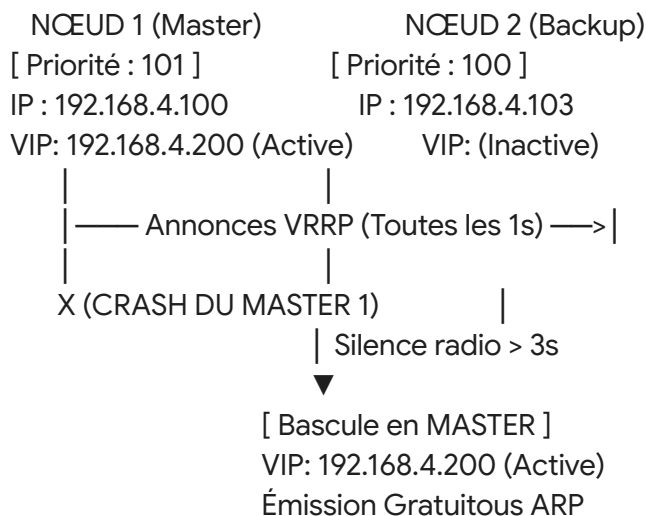
1. Élection du Master :

- Les nœuds s'échangent des trames d'annonce VRRP en multicast sur l'adresse réseau 224.0.0.18 (ou en unicast direct) toutes les secondes.
- Le nœud possédant la plus haute **priorité** numérique (priority 101 vs priority 100) s'attribue l'état **MASTER** et monte l'adresse VIP sur son interface réseau.

2. Détection de défaillance et bascule (*Failover*) :

- Si le nœud MASTER cesse d'émettre ses annonces (panne matérielle, coupure câble, extinction), le nœud BACKUP constate le silence radio au bout de ³ battements de cœur (*advertisement intervals*).
- Le BACKUP bascule instantanément dans l'état MASTER, s'approprie l'adresse IP virtuelle et émet un paquet **Gratuitous ARP (GARP)** sur le réseau local.
- Le commutateur (Switch) met à jour sa table CAM : l'adresse IP virtuelle est désormais routée vers la nouvelle machine **en moins d'une seconde, sans interruption pour les clients**.

BASCULE AUTOMATIQUE VRRP



B. Couplage Keepalived & HAProxy : Le script de surveillance (vrrp_script)

Par défaut, Keepalived ne surveille que la santé matérielle du serveur hôte. Si le système d'exploitation tourne mais que le processus haproxy crashe, Keepalived conserverait la VIP sur

un serveur incapable de router le trafic !

La directive `vrp_script` résout ce problème en exécutant périodiquement une commande d'audit (`killall -0 haproxy` ou `pidof haproxy`). Si HAProxy ne répond plus, Keepalived dégrade dynamiquement la priorité du serveur (`weight -20`), provoquant une bascule propre de la VIP vers le répartiteur de secours.

4. Administration à chaud : La Socket Runtime API de HAProxy

En exploitation de production (MCO), il est formellement proscrit de redémarrer le démon HAProxy (`systemctl restart haproxy`) pour modifier l'état d'un serveur. Même si le rechargement (`reload`) est généralement propre, il invalide des contextes mémoire et coupe des sockets persistantes sous très forte charge.

HAProxy intègre une **Runtime API** pilotable via un socket UNIX local (`/run/haproxy/admin.sock`).

A. Les états opérationnels d'un serveur backend

Un serveur déclaré dans HAProxy peut adopter trois états distincts :

1. **READY (Opérationnel)** : Le serveur reçoit normalement sa part du trafic selon l'algorithme d'équilibrage.
2. **DRAIN (Purge des sessions)** : Le serveur **ne reçoit plus aucune nouvelle connexion**, mais continue de traiter les sessions utilisateurs déjà établies (persistance par cookies). C'est le mode préparatoire obligatoire avant toute opération de maintenance.
3. **MAINT (Maintenance totale)** : Le serveur est immédiatement coupé de tout trafic, y compris les requêtes persistantes. Les contrôles de santé passent en mode passif.

B. Pilotage via l'utilitaire socat

Depuis la ligne de commande du système hôte, l'outil `socat` (`SOcket CAT`) transmet des directives instantanées à HAProxy :

```
# 1. Passer un serveur en mode vidage (drainage avant patch)
echo "set server web_backend/node1 state drain" | sudo socat stdio /run/haproxy/admin.sock
```

```
# 2. Sortir totalement un serveur du pool (maintenance)
echo "set server web_backend/node1 state maint" | sudo socat stdio /run/haproxy/admin.sock
```

```
# 3. Réintégrer le serveur en production après validation
echo "set server web_backend/node1 state ready" | sudo socat stdio /run/haproxy/admin.sock
```

```
# 4. Consulter l'état en direct et le nombre de connexions résiduelles
echo "show stat" | sudo socat stdio /run/haproxy/admin.sock | cut -d',' -f1,2,18,57
```

5. Mémo — Haute Disponibilité

A. Fichier type HAProxy (/etc/haproxy/haproxy.cfg)

```
global
    log /dev/log local0
    log /dev/log local1 notice
    chroot /var/lib/haproxy
    user haproxy
    group haproxy
    daemon
    # Socket d'administration Runtime (Droits lecture/écriture pour root)
    stats socket /run/haproxy/admin.sock mode 660 level admin expose-fd listeners
    stats timeout 30s

defaults
    log global
    mode http
    option httplog
    option dontlognull
    timeout connect 5000ms
    timeout client 50000ms
    timeout server 50000ms
    errorfile 400 /etc/haproxy/errors/400.http
    errorfile 500 /etc/haproxy/errors/500.http

# Interface d'écoute publique (VIP ou Wildcard)
frontend http_in
    bind 0.0.0.0:80
    mode http
    # Injection de l'en-tête IP client réel pour les logs Nginx
    option forwardfor
    default_backend web_backend

# Pool de serveurs web de production
backend web_backend
    mode http
    balance roundrobin
    # Sonde de santé applicative : exige une réponse HTTP 2xx ou 3xx
    option httpchk GET / HTTP/1.1\r\nHost:\ localhost
    server node1 192.168.4.101:80 check inter 2000 fall 3 rise 2
```

```
server node2 192.168.4.102:80 check inter 2000 fall 3 rise 2
```

```
# Console Web de supervision HAProxy
```

```
listen stats
```

```
bind 0.0.0.0:8404
```

```
mode http
```

```
stats enable
```

```
stats uri /
```

```
stats refresh 5s
```

```
stats auth admin:AdminMCO2026
```

B. Fichier type Keepalived (/etc/keepalived/keepalived.conf)

```
global_defs {
```

```
router_id HAPROXY_MASTER
```

```
enable_script_security
```

```
script_user root
```

```
}
```

```
# Script de vérification de l'état du démon local
```

```
vrrp_script check_haproxy {
```

```
script "/usr/bin/killall -0 haproxy"
```

```
interval 2
```

```
weight -20
```

```
fall 2
```

```
rise 2
```

```
}
```

```
vrrp_instance VI_STATIC {
```

```
state MASTER
```

```
interface eth0
```

```
virtual_router_id 51
```

```
priority 101
```

```
advert_int 1
```

```
authentication {
```

```
auth_type PASS
```

```
auth_pass SecretMCO2026
```

```
}
```

```
virtual_ipaddress {
```

```
192.168.4.200/24 dev eth0 label eth0:vip
```

```
}  
  
track_script {  
    check_haproxy  
}  
}
```

C. Boîte à outils CLI d'astreinte (HA & Sockets)

1. Vérifier la présence physique de l'IP virtuelle flottante

```
ip -c a show dev eth0
```

2. Valider la syntaxe du fichier HAProxy sans impacter le service

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
```

3. Surveiller les annonces VRRP en direct sur le réseau local

```
sudo tcpdump -nn -i eth0 vrrp
```

4. Forcer une bascule de test en coupant temporairement HAProxy

```
sudo systemctl stop haproxy
```

(Constater la disparition immédiate de l'alias eth0:vip au profit du backup)

ORCHESTRATION AVANCÉE, ROLLING-UPDATE & MCO ZÉRO-DOWNTIME

1. Problématique industrielle : Le risque de l'exécution simultanée

Par défaut, lorsqu'un administrateur exécute un playbook Ansible :

```
ansible-playbook -i inventory.ini deploy_patch.yml
```

Le moteur traite l'intégralité des nœuds en parallèle (par tranches définies par forks = 5 dans ansible.cfg). Si ce parallélisme massif est un atout pour le déploiement initial (*Day-0*), il constitue **une cause majeure d'interruption de service en exploitation (Day-2)** :

1. **Indisponibilité totale (Downtime)** : Si la tâche ordonne systemctl restart nginx,

l'ensemble des serveurs frontaux redémarre à la même seconde. Aucun serveur n'étant disponible pour répondre aux requêtes entrantes, les clients subissent une erreur HTTP 502 Bad Gateway.

2. **Propagation foudroyante d'une anomalie** : Si une régression de syntaxe s'est glissée dans un gabarit de configuration ou si un paquet de mise à jour s'avère instable, la panne est appliquée à 100% du parc avant que la moindre alerte Nagios n'ait pu être émise.
3. **Perte de contrôle du retour arrière (*Rollback*)** : En cas d'échec au milieu du déploiement, le parc se retrouve fragmenté dans un état hybride non documenté.

+-----+		
	COMPARAISON DES STRATÉGIES DE DÉPLOIEMENT EN MCO	
+-----+		
	DÉPLOIEMENT PARALLÈLE BRUT	
	(Interruption de service)	
+-----+		
	ROLLING-UPDATE ZÉRO-DOWNTIME	
	(Continuité de service 100%)	
+-----+		
	Node 1 : [Reboot / Redémarrage]	
	Node 2 : [Reboot / Redémarrage]	
	Node 1 : Sortie pool LB (Drain)	
	Node 2 : ACTIF (Reçoit 100% du trafic)	
	Node 1 : Mise à jour + Test santé local	
	Node 2 : Traitement des tâches	
	RÉSULTAT : COUPURE DE PRODUCTION	
	Erreur client : HTTP 502 / Drop	
	Node 1 : Réintégration pool LB (Ready)	
	Node 2 : Traitement des tâches	
	RÉSULTAT : ZÉRO COUPURE UTILISATEUR	
+-----+		

2. Les directives d'orchestration continue d'Ansible

Pour transformer une exécution brute en un déploiement progressif sans coupure (*Zero-Downtime Rolling Update*), Ansible fournit un ensemble de directives de contrôle de flux.

A. Traitement par lots séquentiels : **serial**

La directive **serial** s'applique au niveau du *Play*. Elle contraint Ansible à diviser l'inventaire en sous-groupes successifs. Le lot suivant ne démarre que lorsque le lot courant a exécuté **avec succès l'intégralité des tâches du play**.

- **Entier fixe (serial: 1)** : Les serveurs sont traités un par un. C'est la règle obligatoire pour les grappes à deux nœuds (node1, node2) ou les clusters de bases de données nécessitant le maintien strict d'un quorum.
- **Pourcentage (serial: "20%")** : Traite les serveurs par fractions de la flotte (ex. 20 serveurs à la fois sur un parc de 100).
- **Stratégie Canari progressive (serial: [1, 5, "50%"])** :
 1. Traite d'abord un serveur unique (**le canari**) pour valider l'absence de régression.
 2. Si le canari réussit, accélère en traitant 5 machines.

3. Déploie enfin les 50% restants par blocs.

B. Seuil d'interruption d'urgence : `max_fail_percentage`

Par défaut, Ansible poursuit l'exécution tant qu'au moins un serveur du lot survit. Dans un déploiement progressif, cela signifierait qu'une mise à jour défectueuse détruirait un serveur, puis passerait au suivant.

La directive `max_fail_percentage: 0` impose l'**avortement immédiat du playbook dès la première défaillance constatée**. Les serveurs des lots suivants restent intacts et continuent de porter la production.

C. Déport d'action contextuel : `delegate_to`

Lorsqu'Ansible traite une tâche pour `node1`, il exécute le module sur `node1`. Or, pour réaliser une opération propre, `node1` doit d'abord être retiré de la table du répartiteur de charge HAProxy situé sur le Master.

`delegate_to` permet d'exécuter une tâche sur un autre serveur (le Master ou un pare-feu) **tout en conservant le contexte, les faits et les variables de l'hôte cible en cours de traitement** :

```
- name: Retirer node1 du pool HAProxy avant mise à jour
  ansible.builtin.shell:
    cmd: echo "set server web_backend/{{ inventory_hostname }} state maint" | socat stdio
        /run/haproxy/admin.sock
  delegate_to: 192.168.4.100 # Exécuté sur le Master hébergeant HAProxy
```

D. Tâche unitaire dédoublée : `run_once`

Indique à Ansible qu'une tâche ne doit être exécutée qu'une seule fois pour l'ensemble du lot, sur le premier serveur disponible.

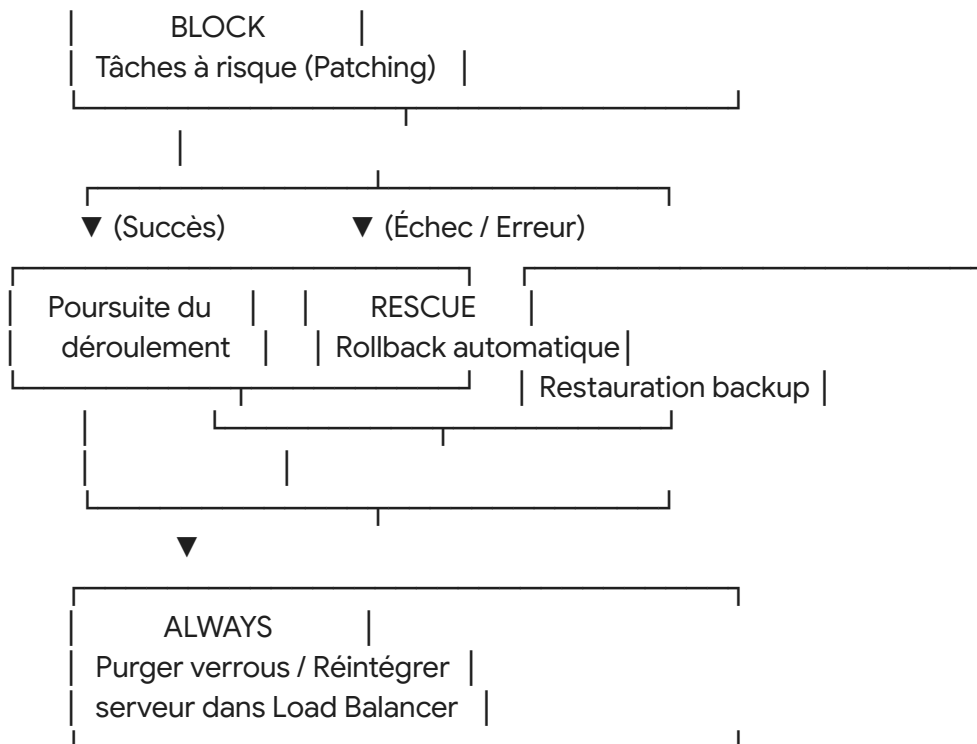
- *Cas d'usage typique* : Lancer une migration de schéma de base de données SQL ou purger un cache partagé NFS sans que 10 serveurs applicatifs ne lancent la même requête concurrente.

3. Gestion des exceptions et stratégies de Rollback : `block`, `rescue` & `always`

Les opérations d'exploitation critiques (patching système, remplacement de fichiers de configuration maîtres) doivent être protégées par une structure équivalente au `try / catch / finally` du développement logiciel.

STRUCTURE BLOCK / RESCUE / ALWAYS

```
|_____|
```



1. **block** : Encapsule la suite d'instructions nominales à risque (copie du nouveau binaire, injection du gabarit, redémarrage).
2. **rescue** : Ne s'exécute **que si une tâche du bloc parent a échoué (failed)**. C'est le réceptacle des actions correctives d'urgence :
 - Restauration de la sauvegarde précédente (backup.conf → active.conf).
 - Redémarrage de l'ancienne version fonctionnelle.
 - Émission d'une alerte critique.
3. **always** : S'exécute **dans 100 % des cas**, que le bloc ait réussi ou qu'il soit passé par le rescue :
 - Suppression des verrous de maintenance temporaires (/tmp/upgrade.lock).
 - Réintégration inconditionnelle de la machine dans le pool du répartiteur de charge pour ne pas dégrader la capacité de traitement du cluster.

4. Le cycle d'un Rolling-Update MCO

Le schéma ci-dessous détaille la séquence industrielle rigoureuse appliquée à chaque serveur pour garantir le zéro-downtime absolu :

ÉTAPE 1 : DRAINAGE LOAD BALANCER

Master (HAProxy) bascule le serveur en état DRAIN / MAINT via socket UNIX.



ÉTAPE 2 : ATTENTE DE VIDAGE SESSIONS

Temporisation de 5 secondes pour laisser terminer les requêtes HTTP en vol.



ÉTAPE 3 : APPLICATION DU CORRECTIF / PATCH

Mise à jour du système, déploiement de configuration et redémarrage démon.



ÉTAPE 4 : QUALIFICATION LOCALE (HEALTH CHECK)

Le Master exécute une requête HTTP directe sur l'IP du serveur sans passer par le LB.
Contrôle strict : code HTTP 200 attendu.



▼ (Succès HTTP 200)

▼ (Échec HTTP 500 / Timeout)

ÉTAPE 5a : RÉINTÉGRATION

HAProxy repasse le serveur en état READY. Trafic réactivé.

ÉTAPE 5b : DÉCLENCHEMENT DU ROLLBACK

Restauration immédiate de l'ancienne conf.
Arrêt immédiat du playbook (max_fail).



ÉTAPE 6 : PASSAGE AU NŒUD SUIVANT

5. Mémo — Rolling-Update & Rollback

A. Playbook complet de mise à jour Zéro-Downtime (rolling_update.yml)

```
- name: "Maintenance applicative et Rolling-Update sans coupure de service"
  hosts: production_nodes
  become: true
  serial: 1          # Traitement strict machine par machine
  max_fail_percentage: 0  # Arrêt immédiat de la chaîne au premier échec
```

vars:

```
lb_master_ip: "192.168.4.100"
haproxy_socket: "/run/haproxy/admin.sock"
service_port: 80
```

tasks:

```
# -----
# 1. SORTIE DU POOL DE RÉPARTITION (ISOLATION)
# -----
```

```
- name: "HAProxy : Sortir {{ inventory_hostname }} du pool actif (MAINT)"
  ansible.builtin.shell:
    cmd: echo "set server web_backend/{{ inventory_hostname }} state maint" | socat stdio {{
haproxy_socket }}
  delegate_to: "{{ lb_master_ip }}"
  changed_when: true
```

```
- name: "Drainage : Attente de la clôture des transactions en vol (5 secondes)"
  ansible.builtin.pause:
    seconds: 5
```

```
# -----
# 2. OPÉRATION À RISQUE ET RATRAPAGE D'ERREURS (BLOCK / RESCUE / ALWAYS)
# -----
```

```
- name: "Exécution des opérations de maintenance avec rollback automatique"
  block:
    - name: "Création d'une copie de sauvegarde locale de la configuration active"
      ansible.builtin.copy:
        src: /etc/nginx/sites-available/custom_site.conf
        dest: /etc/nginx/sites-available/custom_site.conf.bak
        remote_src: true
        mode: '0644'
```

```
- name: "Déploiement de la nouvelle configuration Nginx"
  ansible.builtin.template:
    src: templates/custom_site.conf.j2
    dest: /etc/nginx/sites-available/custom_site.conf
    owner: root
    group: root
    mode: '0644'
```

```
- name: "Validation de la syntaxe Nginx avant redémarrage (Dry-run)"
  ansible.builtin.command:
    cmd: nginx -t
  changed_when: false
```

```
- name: "Application de la configuration : Rechargement du service web"
  ansible.builtin.systemd:
    name: nginx
    state: reloaded
```

```
- name: "Qualification unitaire : Contrôle de santé HTTP local depuis le Master"
  ansible.builtin.uri:
```

```
url: "http://{{ ansible_default_ipv4.address }}:{{ service_port }}/"
status_code: 200
timeout: 3
delegate_to: "{{ lb_master_ip }}"
```

rescue:

- name: "ALERTE D'INCIDENT : Déclenchement du protocole de Rollback d'urgence"

ansible.builtin.debug:

msg: "CRITICAL: Défaillance détectée sur {{ inventory_hostname }}. Restauration de la sauvegarde."

- name: "Restauration de la configuration stable précédente"

ansible.builtin.copy:

src: /etc/nginx/sites-available/custom_site.conf.bak

dest: /etc/nginx/sites-available/custom_site.conf

remote_src: true

mode: '0644'

- name: "Redémarrage du service Nginx en configuration de secours"

ansible.builtin.systemd:

name: nginx

state: restarted

- name: "Interruption du déploiement pour protéger le reste du parc"

ansible.builtin.fail:

msg: "Le déploiement a échoué sur {{ inventory_hostname }}. Rollback exécuté, arrêt d'urgence."

always:

3. RÉINTÉGRATION SYSTÉMATIQUE DANS LE RÉPARTITEUR DE CHARGE

- name: "HAProxy : Réintégrer {{ inventory_hostname }} dans le pool actif (READY)"

ansible.builtin.shell:

cmd: echo "set server web_backend/{{ inventory_hostname }} state ready" | socat stdio {{ haproxy_socket }}

delegate_to: "{{ lb_master_ip }}"

changed_when: true

- name: "Purger les fichiers temporaires de verrouillage"

ansible.builtin.file:

path: /tmp/maintenance.lock

state: absent

B. Contrôle d'idempotence avancée (changed_when & failed_when)

Pour préserver la conformité des audits, les modules bas niveau doivent être strictement maîtrisés :

1. Audit passif sans modification d'état (ne renvoie JAMAIS changed)

- name: Vérifier la table des processus Nginx

ansible.builtin.command: pgrep -c nginx

register: nginx_proc_count

changed_when: false

2. Modification conditionnée par l'analyse textuelle de sortie (stdout)

- name: Exécuter le script de synchronisation de base de données

ansible.builtin.command: /usr/local/bin/sync_cluster.sh

register: sync_out

changed_when: "'CHANGES_APPLIED' in sync_out.stdout"

failed_when:

- sync_out.rc != 0

- "'NON_FATAL_WARNING' not in sync_out.stderr"

6. Synthèse : MCO

Domaine MCO	Outil / Composant	Rôle opérationnel	Indicateur
-------------	-------------------	-------------------	------------

	clé		d'astreinte
Équilibrage de charge (L7)	HAProxy	Répartition des flux, healthchecks, terminaison HTTP.	Console Web :8404, taux d'erreur 502/503.
Résilience réseau (L3)	Keepalived (VRRP)	IP virtuelle flottante (VIP), bascule automatique < 1s.	Trame GARP, présence de l'alias d'interface eth0:vip.
Contrôle d'orchestration	Ansible serial: 1	Découpage des opérations par lots étanches.	Progression pas-à-pas, pas de coupure globale.
Sécurité d'exécution	Block / Rescue / Always	Filet de sécurité, retour arrière immédiat en cas d'erreur.	Événements rescue, statut de rollback dans les logs.